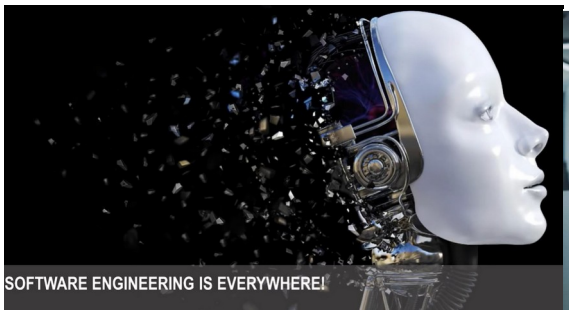
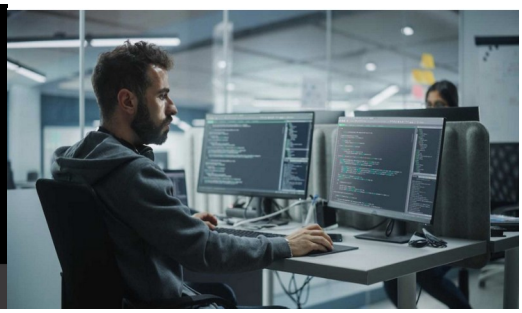
WHAT IS SOFTWARE ENGINEERING?





methods.

1.1. Definition:

Software engineering is a concept in and of itself, but to better understand it, you need to know what each part of the term means before you can fully understand how they operate together. It can be difficult to understand, even though it does seem straightforward. That is because the pieces are more complicated than many believe - and working with software engineering for an application is difficult and time-consuming. Software engineering has two parts: software and engineering.

Software is a collection of codes, documents, and triggers that does a specific job and fills a specific requirement.

Engineering is the development of products using best practices, principles, and

Software engineering is a detailed study of engineering to the design, development and maintenance of software. Software engineering was introduced to address the issues of low-quality software projects. Problems arise when a software generally exceeds timelines, budgets, and reduced levels of quality. It ensures that the application is built consistently, correctly, on time and on budget and within requirements. The demand of software engineering also emerged to cater to the immense rate of change in user requirements and environment on which application is supposed to be working.

What is Software Engineering?

Software engineering is the branch of computer science that deals with the design, development, testing, and maintenance of software applications. Software engineers apply engineering principles and knowledge of programming languages to build software solutions for end users.

Software engineering is a detailed study of engineering to the design, development and maintenance of software. Software engineering was introduced to address the issues of low-quality software projects. Problems arise when a software generally exceeds timelines, budgets, and reduced levels of quality. It ensures that the application is built consistently, correctly, on time and on budget and within requirements. The demand of software engineering also emerged to cater to the immense rate of change in user requirements and environment on which application is supposed to be working.

Software Engineering is the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software.

1. Software engineering includes a variety of techniques, tools, and methodologies, including requirements analysis, design, testing, and maintenance.
2. It is a rapidly evolving field, and new tools and technologies are constantly being developed to improve the software development process.
3. By following the principles of software engineering and using the appropriate tools and methodologies, software developers can create high-quality, reliable, and maintainable software that meets the needs of its users.
4. Software Engineering is mainly used for large projects based on software systems rather than single programs or applications.
5. The main goal of Software Engineering is to develop software applications for improving quality, budget, and time efficiency.
6. Software Engineering ensures that the software that has to be built should be consistent, correct, also on budget, on time, and within the required requirements.

1.2. Software engineering roles and responsibilities

The roles and responsibilities of a software engineer can vary depending on the specific job, industry, and company. Some common day-to-day responsibilities include:

- **Designing and developing software systems and apps**: Designing and developing systems and apps involves writing code and debugging errors. Software engineers also design ways to test new code—either their own or code written for other programs that might be used with their systems—to ensure quality and performance control.
- **Collaborating with other engineers and stakeholders**: This type of collaboration includes working with other software engineers, project managers, and clients to ensure that software products meet their needs. Collaboration takes place throughout the project from the outset (when software requirements are defined) through completion (when the product is evaluated to make sure it meets its defined goals).
- **Maintaining software systems applications**: This involves fixing bugs, updating software, and ensuring that software products continue to function properly over time. The dynamic nature of evolving technology requires software engineers to continuously check on the health of systems and apps.

Applications of software engineering

Software engineering applications are practically unlimited. In fact, software engineering has grown so fast in the last several decades that it has become tightly connected with all other areas of computer science (e.g., artificial intelligence, parallel processing, computer graphics, human-computer interaction, database management systems, and many others) as well as

with some areas outside the traditional computer domain (such as cognitive sciences and visual arts).

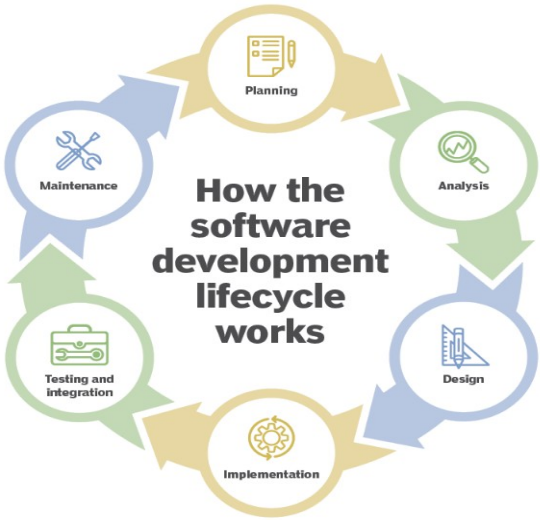
Challenges in software engineering

Current challenges in software engineering research and development include:

- Dealing with the increased complexity of software required in new applications
- The diversity of software systems that need to communicate/operate with each other
- The need to develop trustworthy software
- Increased market demands for a quick turnaround from concept to deployment and operation
- The quest for increased efficiency in component-based reuse and automatic code generation

1.3. Types of software engineering

Even though a software engineer usually manages many coding projects, software engineering entails more than just writing code for the software. In reality, software engineering encompasses every phase of the software development lifecycle (SDLC), from planning the budget to analysis, design, development, software testing, integration, quality and retirement.



Most software engineering tasks can be broken into the following three categories:

- **Operational software engineering.** It includes all decisions and tasks pertaining to how the software will perform within a computer system. This may include anything related to the software budget, the way teams and users will interact with the software and any potential risks such as those associated with defective and outdated software.
- **Transitional software engineering.** This type of software engineering entails duties related to the software's adaptability and scalability when it's moved outside of its initial setting.

- **Software engineering maintenance.** It entails activities connected to enhancing and debugging current software to account for environmental changes, new technologies, bugs and risk factors that might have been disregarded during a previous development cycle.

Let's check if we have understood

Read carefully the following questions and answer.

1. According to the text, what are the two core components of software engineering, and how does understanding each part contribute to a complete understanding of the field?

_____
_____
_____

2. The text mentions that software engineering was introduced to address issues with software projects. What specific problems related to quality, budget, and timelines does software engineering aim to solve?

_____
_____
_____

3. Explain how software engineering differs when applied to large software systems versus single programs or applications.

_____
_____
_____

4. The document outlines several roles and responsibilities of a software engineer. Describe three of these responsibilities and explain why each is crucial to the success of a software project.

_____
_____
_____

5. According to the provided information, what are some of the current challenges in software engineering research and development? Choose two challenges and elaborate on why they are significant.

_____
_____
_____

Indicate whether the following statements are true or false, based on the provided text

1. **True or False**: Software engineering primarily focuses on writing code for software applications.

2. **True or False**: Software engineering is only applicable to projects within the traditional computer domain.

3. **True or False**: One of the main goals of software engineering is to reduce the rate of change in user requirements.

4. **True or False**: Maintaining software systems involves fixing bugs, updating software, and ensuring proper function over time.

5. **True or False**: Automatic code generation decreases efficiency in component-based reuse.

READING COMPREHENSION

Read the following text , then answer the questions.

SOFTWARE ENGINEERING BASICS

The true work of software engineering begins before the product has even been designed – and the software engineering basics dictate that it continues long after the "work" has been completed. It all begins with a thorough and complete understanding of what your software needs to have this includes what the software needs to do, the system in which it needs to operate, and all of the security that it entails. Security is one of the software engineering basics because it is so essential to all aspects of development. Without tools to help you better understand how your code is being built and where any security problems may fall, your team can easily become lost in the development stage.

Software engineering design basics require creating the instructions for the computer and the systems. Much of this will take place at the coding level by professionals who have comprehensive training. Still, it is important to understanding that software engineering isn't always a linear process, which means that it requires thorough vetting once it has been completed.

Not all software requires software engineering. Simplistic games or programs that are used by consumers may not need engineering, depending on the risks associated with them. Almost all companies do require software engineering because of the high-risk information that they store and security risks that they pose.

Software engineering helps to create customized, personalized software that should look into vulnerabilities and risks before they even emerge. Even when the software engineering principles of safety aren't required, it can also help to reduce costs and improve customer experience.

Types of Software Engineering

Software engineering studies the design, development, and maintenance of software as an umbrella definition. Still, there are different types of software engineering that a company or product may need. Problems tend to emerge when software is low-quality or isn't properly vetted before deployment.

There has been a lot of demand for software engineers because of the rate of change in user requirements, statutes, and the platforms we use. Software engineering works on a few different levels: *Operational Software Engineering:* Software engineering on the operational level focuses on how the software interacts with the system, whether or not it is on a budget, the usability, the functionality, the dependability, and the security.

Transitional Software Engineering: This type focuses on how software will react when it is changed from one environment to another. It typically takes some scalability or flexibility in the development. *Software Engineering Maintenance:* Recurrent software engineering focuses on how the software functions within the existing system, as all parts of it change.

Software engineering functions at all parts of the software development lifecycle, including analysis, design, development, testing, integration, implementation, maintenance, and even retirement.

It is important to understand that software engineering isn't a new practice, but it is constantly changing and can feel new on a regular basis. Software is used in everything around us, so it is important to ensure that all software is working properly. If it does not, it can result in loss of money, loss of reputation, and even in some cases, loss of life.

PRACTICE

Match the following words with their definitions*:*

1. High technology	a) the range of operations that can be run on a computer or other electronic system
2. engineering	b) come into existence or greater prominence
3. functionality	c) take or use another instead of
4. change	d) instructions for a computer in some programming language, often machine language
5. emerge	e) advanced technological development, especially in electronics
6. code	f) a field of study or activity concerned with modification or development in a particular area
7. application developer	g) a position or stage on a scale of quantity, extent, rank, or quality
8. level	h) a person who writes computer programs to meet specific requirements